

本讲内容

√ 序列模式

- u 序列模式简介
- u GSP算法
- u PrefixSpan算法

序列模式简介

- (Temporal) order is important in many situations
 - Time-series databases and sequence databases
 - Frequent patterns → (frequent) sequential patterns
- Applications of sequential pattern mining
 - Customer shopping sequences:
 - First buy computer, then CD-ROM, and then digital camera, within 3 months.
 - Medical treatment, natural disasters (e.g., earthquakes), science & engineering processes, stocks and markets, telephone calling patterns, Weblog click streams, DNA sequences and gene structures

序列模式简介

- Given a set of sequences, find the complete set of frequent subsequences

A sequence database

SID	sequence
10	<a(<u>abc</u>)(a <u>c</u>)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(<u>ab</u>)(df) <u>c</u> b>
40	<eg(af)cbc>

A sequence: <(ef)(ab)(df)c**b**>



An element may contain a set of items. Items within an element are unordered and we list them alphabetically.

<a(bc)dc> is a subsequence of <a(abc)(ac)d(cf)>

Given support threshold $min_sup = 2$, <(ab)c> is a sequential pattern

序列模式简介

- √ **项目集(Itemset)**: 是各种项目组成的集合
- √ **序列(Sequence)**: 是不同项目集(ItemSet)的**有序**排列, 序列s可以表示为 $s = \langle s_1 s_2 \dots s_l \rangle$, $s_j (1 \leq j \leq l)$ 为项目集(Itemset), 也称为序列s的元素
- √ **序列的长度**: 一个序列所包含的项目集(ItemSet)的个数。

序列模式简介

√ 设 $\alpha = \langle a_1 a_2 \dots a_n \rangle$, $\beta = \langle b_1 b_2 \dots b_m \rangle$, 如果存在整数 $1 \leq j_1 < j_2 < \dots < j_n \leq m$, 使得 $a_1 \subseteq b_{j_1}$, $a_2 \subseteq b_{j_2}$, $\dots$, $a_n \subseteq b_{j_n}$, 则称序列 α 为序列 β 的**子序列**, 又称序列 β **包含** 序列 α , 记为 $\alpha \subseteq \beta$

u 例如, 序列 $\langle (3)(4\ 5)(8) \rangle$ 是序列 $\langle (7)(3\ 8)(9)(4\ 5\ 6)(8) \rangle$ 的子序列。

u 但, $\langle (3)(5) \rangle$ 不是 $\langle (3\ 5) \rangle$ 的子序列, 反之亦然。

√ 给定一个序列集合, 如果序列 s 不包含于任何一个其它的序列中, 则称 s 是最大的 (Maximal Sequence)。

项集与序列

项集数据

TID	itemsets
10	a, b, d
20	a, c, d
30	a, d, e
40	b, e, f

序列数据

SID	sequences
10	<a(<u>abc</u>)(<u>ac</u>)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(<u>ab</u>)(df) <u>cb</u> >
40	<eg(af)cbc>

- √ 一条项集数据是一个集合，如{a, b, d}，没有顺序关系
- √ 一条序列数据是由多个项集依次排成的序列，项集之间有顺序关系
- √ 如<a(abc)(ac)d(cf)>表示<{a}{a, b, c}{a, c}{d}{c, f}>
- √ 多元项集需加上括号，无括号的字母视作单项集

子序列

序列数据

SID	sequences
10	<a(<u>abc</u>)(a <u>c</u>)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(<u>ab</u>)(df) <u>c</u> b>
40	<eg(af)cbc>

- √ 若一条序列T的每个字母都能在S中依次找到对应字母
- √ 则称T是S的子序列
- √ 如:
- √ <(ab)c>是序列10和30的子序列
- √ <dc>是序列10、20、30的子序列
- √ <abc>是序列10、40的子序列
- √ <cg>不是任何序列的子序列

小练习

序列数据

SID	sequences
10	<a(<u>abc</u>)(a <u>c</u>)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(<u>ab</u>)(df) <u>c</u> b>
40	<eg(af)cbc>

√ <(ab)d>是序列_____的子序列

√ <cc>是序列_____的子序列

√ <aa>, 支持度是____, _____频繁序列

√ <(ad)e>, 支持度是____, _____频繁序列

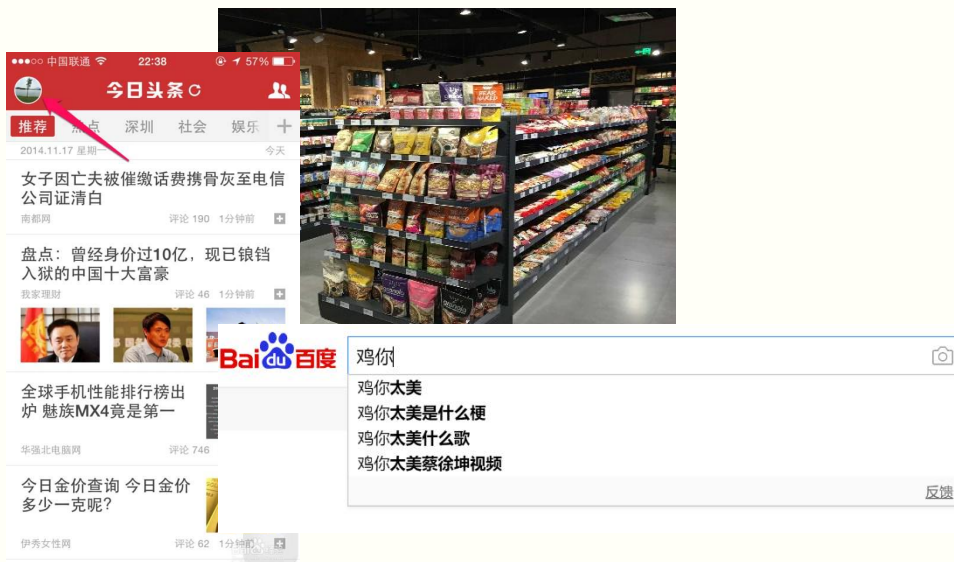
频繁序列

SID	sequence
10	< <u>a</u> (<u>abc</u>)(<u>ac</u>)d(<u>cf</u>)>
20	<(a <u>d</u>)c(bc)(a <u>e</u>)>
30	<(ef)(a <u>b</u>)(d <u>f</u>)c <u>b</u> >
40	<eg(a <u>f</u>)c <u>b</u> c>

- √ 若一条序列T在序列数据集中作为子序列的频率 $\geq \alpha$
- √ 则称序列T是序列数据集的频繁序列
- √ 如:
- √ α 称为阈值, 可以人为设定
- √ *简单起见, 在接下来的所有内容中, α 都设为50%
- √ <(ab)c>, 50% (10、30) 是频繁序列
- √ <ac>, 100% (10、20、30、40) 是频繁序列
- √ <(cf)>, 10% (10) 不是频繁序列
- √ <cab>, 0%, 不是频繁序列

寻找频繁序列

- √ 用途:
- √ 货架摆放
- √ 内容推送
- √ 搜索联想
- √



序列模式简介

√ Customer-sequence

- u All the transactions of a customer, ordered by increasing transaction-time, corresponds to a sequence.
- √ 序列 α 在序列数据库S中的支持数为序列数据库S中包含序列 α 的序列个数，记为 $\text{Support}(\alpha)$
- √ 序列模式挖掘：找出所有最大的频繁子序列。

序列模式简介

- √ **itemset的支持度**: **itemset**作为一个整体, 在整个数据库中出现的比例, 比如, 单次购物时同时购买该**itemset**中所有项目的顾客占全部顾客的百分比
 - u **itemset**的支持度 等于 **1-sequence**的支持度
- √ **litemset (Large itemset)**: **A itemset with minimum support**
- √ **Candidate sequence**
- √ **Large sequence**

Example

Transaction Time	Customer Id	Items Bought
June 10 '93	2	10, 20
June 12 '93	5	90
June 15 '93	2	30
June 20 '93	2	40, 60, 70
June 25 '93	4	30
June 25 '93	3	30, 50, 70
June 25 '93	1	30
June 30 '93	1	90
June 30 '93	4	40, 70
July 25 '93	4	90

Q. How to find the sequential patterns?

Example

以Customer_Id 及 TransactionTime 排序

Customer Id	TransactionTime	Items Bought
1	June 25 '93	30
1	June 30 '93	90
2	June 10 '93	10, 20
2	June 15 '93	30
2	June 20 '93	40, 60, 70
3	June 25 '93	30, 50, 70
4	June 25 '93	30
4	June 30 '93	40, 70
4	July 25 '93	90
5	June 12 '93	90

Transaction
Item

Itemset

Example (cont.)

Customer Id	Customer Sequence
1	{ (30) (90) }
2	{ (10 20) (30) (40 60 70) }
3	{ (30 50 70) }
4	{ (30) (40 70) (90) }
5	{ (90) }

→ Sequence

→ 3-Sequence

$\langle (30) (90) \rangle$ is **supported** by customer 1 and 4

$\langle 30 (40 70) \rangle$ is **supported** by customer 2 and 4

With minimum support of 2 customers:

The **large itemset (litemset)**:

(30), (40), (70), (40 70), (90)



GSP算法

- GSP (Generalized Sequential Pattern) mining
- Outline of the method
 - Initially, every item in DB is a candidate of length-1
 - For each level (i.e., sequences of length- k) do
 - Scan database to collect support count for each candidate sequence
 - Generate candidate length- $(k+1)$ sequences from length- k frequent sequences using Apriori
 - Repeat until no frequent sequence or no candidate can be found
- Major strength: Candidate pruning by Apriori

GSP算法

v **Five phases**

- u **Sort phase**
- u **Large itemset phase**
- u **Transformation phase**
- u **Sequence phase**
- u **Maximal phase**

Apriori

Sort phase

Sort the database with customer-id as the major key and transaction-time as the minor key

Itemset phase

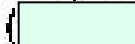
- ✓ Find the large itemset.
- ✓ Itemsets mapping

Large Itemsets	Mapped To
(30)	1
(40)	2
(70)	3
(40 70)	4
(00)	5

Reason: 可把**itemset**当作一个整体对待。比较两个**itemsets**是否相等用的时间是恒定的，在判断某个序列是否包含于另一个序列时可以省时间。

Transformation phase

- ✓ **Deleting non-large itemsets**
- ✓ **Mapping large itemsets to integers**

Customer Id	Original Customer Sequence	Transformed Customer Sequence	After Mapping
1	((30) (90))	{{(30)} {(90)}}	{{1} {5}}
2	(( (30) (40  70))	{{(30)} {(40), (70), (40 70)}}	{{1} {2, 3, 4}}
3	((30  70))	{{(30), (70)}}	{{1, 3}}
4	((30) (40 70) (90))	{{(30)} {(40), (70), (40 70)} {(90)}}	{{1} {2, 3, 4} {5}}
5	((90))	{{(90)}}	{{5}}

Sequence phase

- ✓ Use the set of **itemsets** to find the **large** sequence (frequent sequence).

Maximal phase

- ✓ Find the maximum sequences among the set of large sequences.
- ✓ In some algorithms, this phase is combined with the sequence phase.

Maximal phase

✓ Algorithm:

- u S the set of all large sequences
- u n the length of the **longest** sequence

```
for ( $k = n$ ;  $k > 1$ ;  $k--$ ) do
  foreach  $k$ -sequence  $s_k$  do
    Delete from  $S$  all subsequences of  $s_k$ 
```

AprioriAll算法

- ✓ The basic method to mine sequential patterns
- ✓ Based on the Apriori algorithm.
- ✓ Count **all the large sequences**, including non-maximal sequences.
- ✓ Use **Apriori-generate function** to generate candidate sequence.

Apriori Candidate Generation

- ∨ generate candidates for pass using only the **large sequences** found in the previous pass and then makes a pass over the data to find their support.

Apriori Candidate Generation

✓ Algorithm:

- u L_{k-1} the set of all large $(k-1)$ -sequences
- u C_k the set of candidate k -sequences

Join Phase:

insert into C_k

select $p.\text{itemset}_1, p.\text{itemset}_2, \dots, p.\text{itemset}_{k-1}, q.\text{itemset}_{k-1}$

from $L_{k-1} p, L_{k-1} q$

where $p.\text{itemset}_1 = q.\text{itemset}_1, \dots, p.\text{itemset}_{k-2} = q.\text{itemset}_{k-2}$;

Prung Phase:

forall sequences $c \in C_k$ do

forall $(k-1)$ -subsequences s of c do

if $(s \notin L_{k-1})$ then

delete c from C_k ;

Example

v 关联规则 与 序列模式 生成候选集时的区别

u **join** 序列模式需要关注顺序

u 例如, 有如下三条序列

100 <(10) (20)>

200 <(20) (10)>

200 <(20) (10)>

假设 $\text{minsupp}=1$, (10) (20)都是litemset, 分别map为1, 2
则生成C2时需要同时包含<12>和<21>

可以发现, 序列<(10) (20)>和<(20) (10)>都是所需要的序列模式。如果join时不考虑顺序的话, 将丢掉<(20) (10)>。

AprioriAll (cont.)

```
 $L_1 = \{\text{large 1-sequences}\};$  // Result of litemset phase  
for (  $k=2$ ;  $L_{k-1} \neq \Phi$ ;  $k++$ ) do  
  begin  
     $C_k = \text{New candidate generate from } L_{k-1}$   
    foreach customer-sequence  $c$  in the database do  
      Increment the count of all candidates in  $C_k$  that are contained in  $c$   
     $L_k = \text{Candidates in } C_k \text{ with minimum support.}$   
  End  
Answer=Maximal Sequences in  $L_k$ ;
```

Example

v 关联规则 与 序列模式 对 **itemset** 计数时的区别

u 如果相同的项目集合在同一个序列里出现多次的话，只记一次

u 例如，有如下三条序列

100 <(10) (20) (10)>

200 <(20)>

200 <(20) (30)>

假设 **minsuff=2**，则 **(20)** 是 **litemset**，而 **(10)** 只能算出现了一次，因而不是 **litemset**。

Apriori Candidate Generation

✓ Example: (Customer Sequences)

$\langle \{1\ 5\} \{2\} \{3\} \{4\} \rangle$

$\langle \{1\} \{3\} \{4\} \{3\ 5\} \rangle$

$\langle \{1\} \{2\} \{3\} \{4\} \rangle$

$\langle \{1\} \{3\} \{5\} \rangle$

$\langle \{4\} \{5\} \rangle$

With minimum set to 25%

next step: find the **large 1-sequences**

Example

<{**1** 5}{2}{3}{4}>
<{**1**}{3}{4}{3 5}>
<{**1**}{2}{3}{4}>
 <{**1**}{3}{5}>
 <{4}{5}>

Large 1-Sequence

Sequence	Support
<1>	4
<2>	2
<3>	4
<4>	4
<5>	4

next step: find the large 2-sequences

Example

L_1

Sequence	Support
<1>	4
<2>	2
<3>	4
<4>	4
<5>	4

<{1 5}{2}{3}{4}>
 <{1}{3}{4}{3 5}>
 <{1}{2}{3}{4}>
 <{1}{3}{5}>
 <{4}{5}>

join

pruned

L_2

<12>
<21>
<13>
<31>
<14>
<41>
<15>
<51>
<23>
<32>
<24>
<42>
<25>
<52>
<34>
<43>
<35>
<53>
<45>
<54>

<12>
<21>
<13>
<31>
<14>
<41>
<15>
<51>
<23>
<32>
<24>
<42>
<25>
<52>
<34>
<43>
<35>
<53>
<45>
<54>

Sequence	Support
<1 2>	2
<1 3>	4
<1 4>	3
<1 5>	3
<2 3>	2
<2 4>	2
<3 4>	3
<3 5>	2
<4 5>	2

L_2

<1 2>	2
<1 3>	4
<1 4>	3
<1 5>	3
<2 3>	2
<2 4>	2
<3 4>	3
<3 5>	2
<4 5>	2

join*pruned*

<123>
<132>
<124>
<142>
<125>
<152>
<134>
<143>
<135>
<153>
<145>
<154>
<234>
<243>
<345>
<354>



<123>
<124>
<134>
<135>
<145>
<234>
<345>

 L_3

<1 2 3>	2
<1 2 4>	2
<1 3 4>	3
<1 3 5>	2
<2 3 4>	2

<{1 5}{2}{3}{4}>
 <{1}{3}{4}{3 5}>
 <{1}{2}{3}{4}>
 <{1}{3}{5}>
 <{4}{5}>

L_3 *join*

<1 2 3>	2
<1 2 4>	2
<1 3 4>	3
<1 3 5>	2
<2 3 4>	2



<1234>
<1243>
<1345>
<1354>

*pruned*

1234

 L_4

<1 2 3 4>	2
-----------	---

<{1 5}{2}{3}{4}>

<{1}{3}{4}{3 5}>

<{1}{2}{3}{4}>

<{1}{3}{5}>

<{4}{5}>

Example

Find the **maximal** large sequences

Sequence	Support
<1>	4
<2>	2
<3>	4
<4>	4
<5>	2

Sequence	Support
<1 2 3>	2
<1 2 4>	2
<1 3 4>	3
<1 3 5>	2
<2 3 4>	2



Sequence	Support
<1 2>	2
<1 3>	4
<1 4>	3
<1 5>	3
<2 3>	2
<2 4>	2
<3 4>	3
<3 5>	2
<4 5>	2



Sequence	Support
<1 2 3 4>	2

judgment

**Waste too much time in counting
non-maximal sequence, which is
impossible to be a sequential pattern.**

AprioriSome

- ✓ **It generates candidates for a pass using only the large sequences found in the previous pass.**
- ✓ **Divided into 2 phase:
forward vs. backward**
- ✓ **Advantage:**
Reduce counting time wasted in counting non-maximal ones.

Next function

$$hits = |L_k| / |C_k|$$

```
function next(k: integer)
begin
    if (hitk < 0.666) return k + 1;
    elseif (hitk < 0.75) return k + 2;
    elseif (hitk < 0.80) return k + 3;
    elseif (hitk < 0.85) return k + 4;
    else return k + 5;
end
```

AprioriSome (cont.)

//Forward Phase

$L_1 = \{\text{Large 1-sequences}\}$; // Result of the litemset phase

$C_1 = L_1$; // so that we have a nice loop condition

$Last = 1$; // we last counted C_{last}

for ($k=2$; $C_{k-1} \neq \emptyset$ and $L_{last} \neq \emptyset$; $k++$) **do**

begin

if (L_{k-1} known) **then**

$C_k = \text{New candidates generated from } L_{k-1}$;

Else

$C_k = \text{New candidates generated from } C_{k-1}$;

if ($k == \text{next}(last)$) **then begin**

Foreach customer-sequence c in the database **do**

 Increment the count of all candidates in C_k that are contained in c

$L_k = \text{Candidates in } C_k \text{ with minimum support.}$

$Last = k$;

end

end

AprioriSome (cont.)

//Backward Phase

for ($k--$; $k \geq 1$; $k--$) **do**

if (L_k was not determined in the forward phase) **then begin**

 Delete all sequences in C_k contained in some L_i , $i > k$;

foreach customer-sequence c in D_T **do**

 Increment the count of all candidates in C_k that are contained in c .

L_k = Candidates in C_k with minimum support.

end

else begin // L_k already known

 Delete all sequences in L_k contained in some L_i , $i > k$;

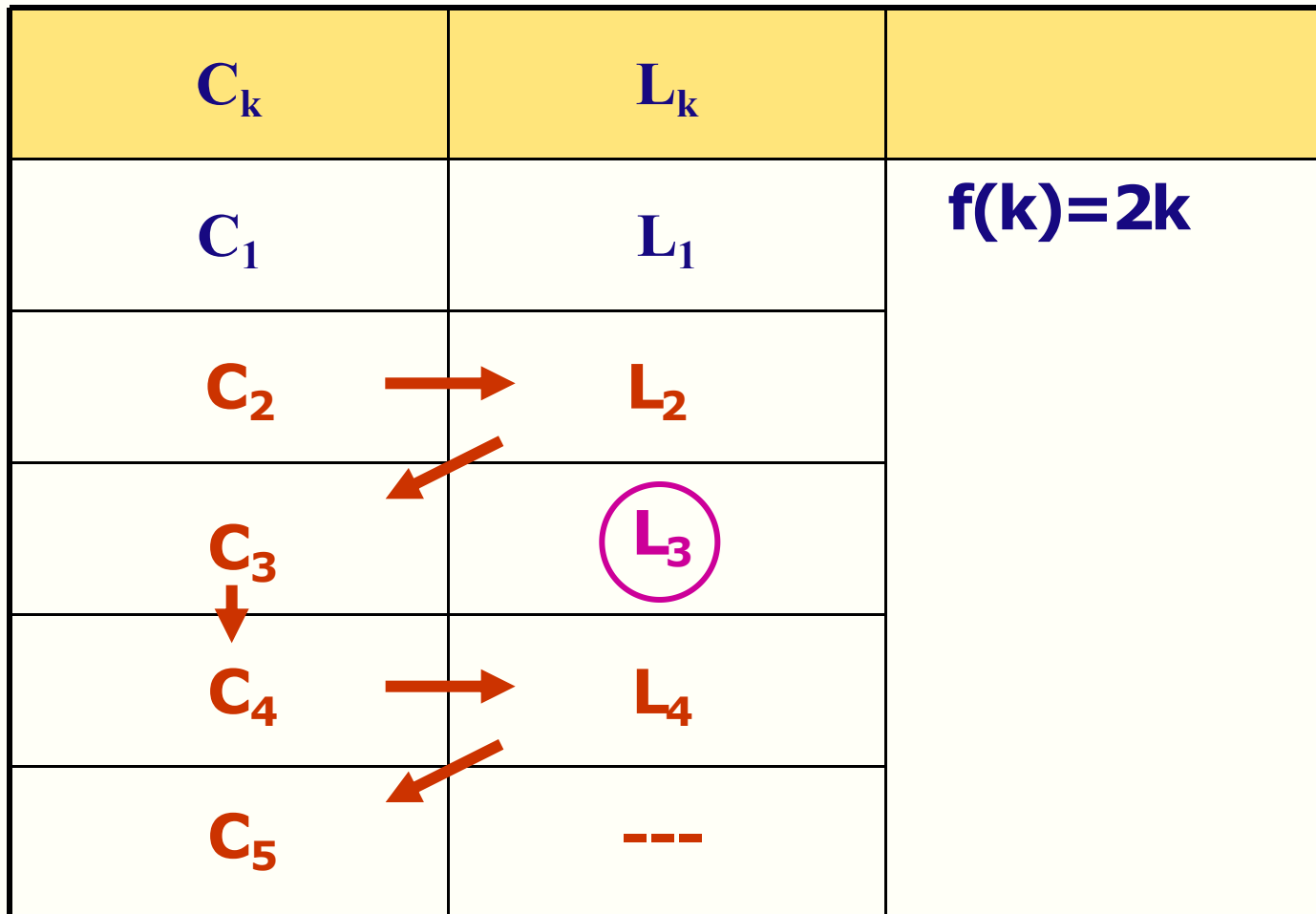
end

end

Answer = $\bigcup_k L_k$;

Example - AprioriSome

C_k	L_k	
C_1	L_1	$f(k)=2k$
C_2	L_2	
C_3	L_3	
C_4	L_4	
C_5	---	



The diagram illustrates the AprioriSome process. It shows a sequence of candidate sets C_k and their corresponding large sets L_k . The function $f(k)=2k$ is used to determine the support of the sets. The sets C_2 , C_3 , C_4 , and C_5 are shown, along with their corresponding L_k sets. The set L_3 is highlighted with a pink circle, indicating it is a large set. The sets C_2 , C_4 , and C_5 are shown with arrows pointing to their corresponding L_k sets, indicating they are not large sets. The set C_3 is shown with an arrow pointing to L_3 , indicating it is a large set. The set C_4 is shown with an arrow pointing to L_4 , indicating it is not a large set. The set C_5 is shown with an arrow pointing to a dashed line, indicating it is not a large set.

So far, we covered

	Read-based	Write-based	Point-based
Association Mining	Apriori[AgSr94]	VIPER[SHSB00] FPGrowth [HaPe00]	
Sub-Graph Mining	AGM[PKDD'00] FSG[ICDM'01]	Mofa[ICDM'02] gSpan[ICDM'02]	
Sequential Pattern Discovery	GSP[AgSr96]	PrefixSpan [PHPC01]	
Iceberg Cube	Apriori[AgSr94]		BUC[BeRa99], H-cubing [HPDW01]

References

- v [AgSr94] R. Agrawal, R. Srikant, "Fast Algorithms for Mining Association Rules", Proc. of the 20th Int'l Conference on Very Large Databases, Santiago, Chile, Sept. 1994.
- v [AgSr96] R. Srikant, R. Agrawal: "Mining Sequential Patterns: Generalizations and Performance Improvements", to appear in *Proc. of the Fifth Int'l Conference on Extending Database Technology (EDBT)*, Avignon, France, March 1996.
- v [BeRa99] Kevin S. Beyer and Raghu Ramakrishnan. "Bottom-Up Computation of Sparse and Iceberg CUBEs". In Proceedings of the ACM SIGMOD International Conference on Management of Data, pages 359-370, June 1999.
- v [HPDW01] J. Han, J. Pei, G. Dong, and K. Wang, "Efficient Computation of Iceberg Cubes with Complex Measures", Proc. 2001 ACM-SIGMOD Int. Conf. on Management of Data (SIGMOD'01), Santa Barbara, CA, May 2001.
- v [HPY00] J. Han, J. Pei, and Y. Yin, ``Mining Frequent Patterns without Candidate Generation`, Proc. 2000 ACM-SIGMOD Int. Conf. on Management of Data (SIGMOD'00), Dallas, TX, May 2000.
- v [HaPe00] J. Han and J. Pei ``Mining Frequent Patterns by Pattern-Growth: Methodology and Implications`, ACM SIGKDD Explorations (Special Issue on Scalable Data Mining Algorithms), 2(2), December 2000.
- v [PHPC01] J. Pei, J. Han, H. Pinto, Q. Chen, U. Dayal, and M.-C. Hsu, "PrefixSpan: Mining Sequential Patterns Efficiently by Prefix-Projected Pattern Growth", Proc. 2001 Int. Conf. on Data Engineering (ICDE'01), Heidelberg, Germany, April 2001.

References

- ✓ [SHSB00] P. Shenoy, J. Haritsa, S. Sudarshan, G. Bhalotia, M. Bawa, D. "[Turbo-Charging Vertical Mining of Large Database](#)." Proc. 2000 ACM-SIGMOD Int. Conf. on Management of Data (SIGMOD'00), Dallas, TX, May 2000.
- ✓ [Zaki98] Mohammed J. Zaki, "[Efficient Enumeration of Frequent Sequences](#)", 7th International Conference on Information and Knowledge Management, pp 68-75, Washington DC, November 1998.
- ✓ [Zaki00] Mohammed J. Zaki, "[Scalable Algorithms for Association Mining](#)", in IEEE Transactions on Knowledge and Data Engineering, Vol. 12, No. 3, pp 372-390, May/June 2000.
- ✓ [Zaki01] Mohammed J. Zaki, "[SPADE: An Efficient Algorithm for Mining Frequent Sequences](#)", in Machine Learning Journal, special issue on Unsupervised Learning (Doug Fisher, ed.), pp 31-60, Vol. 42 Nos. 1/2, Jan/Feb 2001.